Cairo 컴파일 파이프라인 개요

소개

Cairo 컴파일러는 Cairo 소스 코드를 실행 가능한 CASM(Cairo Assembly) 코드로 변환하는 복잡한 다단계 파이프라인을 구현합니다. 이 문서는 Cairo → Sierra → CASM의 전체 변환 과정을 상세히 설명하고, 각 단계의 입력과 출력, 그리고 컴파일러의 핵심 구성 요소들을 분석합니다.

전체 컴파일 파이프라인 개요

Cairo 컴파일 과정은 다음과 같은 주요 단계로 구성됩니다:

```
Cairo 소스 코드 (.cairo)

↓

[파싱 단계]

↓

AST (Abstract Syntax Tree)

↓

[의미 분석 단계]

↓

HIR (High-level IR)

↓

[Lowering 단계]

↓
```

```
MIR (Mid-level IR)
↓
[Sierra 생성 단계]
↓
Sierra IR
↓
[Sierra 최적화 단계]
↓
최적화된 Sierra IR
↓
[CASM 생성 단계]
↓
CASM (Cairo Assembly)
```

컴파일러 메인 함수 분석

1. cairo-compile 바이너리

Cairo 컴파일러의 주요 진입점은 `crates/bin/cairo-compile/src/main.rs` 에 위치합니다:

```
fn main() -> anyhow::Result<()> {
init_logging(log::LevelFilter::Off);
log::info!("Starting Cairo compilation.");
```

```
let  args = Args::parse();

check_compiler_path(args.single_file, &args.path)?;


let  sierra_program = compile_cairo_project_at_path(

&args.path,

CompilerConfig {

replace_ids: args.replace_ids,

inlining_strategy: args.inlining_strategy.into(),

..CompilerConfig::default()

},

)?;


// 출력 처리

match  args.output {

Some(path) => fs::write(path, format!("{sierra_program}"))?,

None => println!("{sierra_program}"),

}


Ok(())

}
```

2. 핵심 컴파일 함수

`compile_cairo_project_at_path` 함수는 컴파일 파이프라인의 핵심입니다:

```
pub fn compile_cairo_project_at_path(
path: &Path,
compiler_config: CompilerConfig<'_>,
) -> Result<Program> {
// 1. 데이터베이스 초기화
let mut db = RootDatabase::builder()
.with_inlining_strategy(compiler_config.inlining_strategy)
.detect_corelib()
.build()?;
// 2. 프로젝트 설정
let main_crate_ids = setup_project(&mut db, path)?;
// 3. 실제 컴파일 실행
compile_prepared_db_program(
&db,
CrateInput::into_crate_ids(&db, main_crate_ids),
compiler_config,
)
}
```

각 컴파일 단계의 상세 분석

1. 파싱 단계 (Cairo → AST)

입력: Cairo 소스 코드 (`.cairo` 파일)

출력: AST (Abstract Syntax Tree)

담당 crate: `cairo-lang-parser`

파싱 단계는 다음 과정을 거칩니다:

- **토큰화 (Lexing)**: 소스 코드를 토큰 스트림으로 변환
- **구문 분석 (Parsing)**: 토큰을 AST 노드로 변환
- **에러 복구**: 구문 오류 발생 시 복구 메커니즘 적용

```
struct SyntaxNode {
kind: SyntaxKind,
children: Vec<SyntaxNode>,
span: TextSpan,
}
```

2. 의미 분석 단계 (AST → HIR)

입력: AST

출력: HIR (High-level Intermediate Representation)

담당 crate: `cairo-lang-semantic`

의미 분석 단계의 주요 작업:

- **타입 체킹**: 모든 표현식과 문장의 타입 검증
- **스코프 해결**: 변수와 함수의 스코프 분석
- **이름 바인딩**: 식별자를 해당 정의에 연결
- **트레이트 해결**: 트레이트 구현과 사용 검증

```
struct SemanticModel {
items: HashMap<ItemId, Item>,
types: HashMap<TypeId, Type>,
functions: HashMap<FunctionId, Function>,
}
```

3. Lowering 단계 (HIR → MIR)

입력: HIR

출력: MIR (Mid-level Intermediate Representation)

담당 crate: `cairo-lang-lowering`

Lowering 단계의 변환 작업:

- **패턴 매칭 해제**: 복잡한 패턴을 단순한 조건문으로 변환
- **제어 흐름 단순화**: 고수준 제어 구조를 기본 블록으로 변환
- **함수 인라이닝**: 설정에 따른 함수 인라인 최적화
- **변수 생명주기 분석**: 메모리 관리를 위한 변수 추적

```
struct LoweredFunction {
signature: Signature,
body: FlatBlocks,
variables: HashMap<VariableId, Variable>,
}
```

4. Sierra 생성 단계 (MIR → Sierra IR)

입력: MIR

출력: Sierra IR

담당 crate: `cairo-lang-sierra-generator`

Sierra 생성 과정:

- **Sierra 명령어 변환**: MIR을 Sierra 명령어로 매핑
- **타입 정보 보존**: 증명 생성을 위한 타입 정보 유지
- **가스 계산 준비**: 실행 비용 계산을 위한 메타데이터 생성
- **함수 의존성 해결**: 호출되는 모든 함수 포함

```
struct Program {
type_declarations: Vec<TypeDeclaration>,
libfunc_declarations: Vec<LibfuncDeclaration>,
statements: Vec<Statement>,
funcs: Vec<Function>,
}
```

5. Sierra 최적화 단계

입력: Sierra IR

출력: 최적화된 Sierra IR

담당 crate: `cairo-lang-sierra-*` (여러 crate)

최적화 기법들:

- **가스 계산 최적화**: 실행 비용 최소화
- **AP 변화 추적**: 메모리 접근 패턴 최적화

- **타입 크기 분석**: 메모리 레이아웃 최적화
- **데드 코드 제거**: 사용되지 않는 코드 제거

6. CASM 생성 단계 (Sierra → CASM)

입력: 최적화된 Sierra IR

출력: CASM (Cairo Assembly)

담당 crate: `cairo-lang-sierra-to-casm`

CASM 생성 과정:

- **명령어 매핑**: Sierra 명령어를 CASM 명령어로 변환
- **레지스터 할당**: 효율적인 레지스터 사용 계획
- **메모리 레이아웃**: 최적화된 메모리 배치
- **가스 메타데이터**: 실행 비용 정보 포함

```
struct CasmProgram {
instructions: Vec<Instruction>,
debug_info: DebugInfo,
}
```

컴파일 설정과 옵션

CompilerConfig 구조체

```
pub struct CompilerConfig<'a> {
pub diagnostics_reporter: DiagnosticsReporter<'a>,
```

```
pub replace_ids: bool,
pub inlining_strategy: InliningStrategy,
pub allowed_libfuncs_list_name: Option<String>,
pub add_statements_functions: bool,
pub add_statements_code_locations: bool,
}
```

주요 컴파일 옵션

1. **replace_ids**: Sierra ID를 사람이 읽기 쉬운 형태로 변경
2. **inlining_strategy**: 함수 인라이닝 전략 설정
   - `Default`: 기본 인라이닝 전략
   - `Avoid`: `inline(always)` 어노테이션이 있는 경우만 인라인
3. **allowed_libfuncs_list_name**: 허용된 라이브러리 함수 목록
4. **add_statements_functions**: 프로파일링을 위한 함수 매핑 정보 추가
5. **add_statements_code_locations**: 커버리지를 위한 코드 위치 정보 추가

명령행 옵션

cairo-compile 바이너리의 주요 옵션들:

```
cairo-compile [OPTIONS] <PATH>

OPTIONS:
-s, --single-file 단일 파일 모드
```

```
-o, --output <OUTPUT> 출력 파일 지정
-r, --replace-ids Sierra ID를 사람이 읽기 쉬운 형태로 변경
-i, --inlining-strategy 인라이닝 전략 설정 (default|avoid)
```

데이터베이스 기반 컴파일 시스템

Cairo 컴파일러는 Salsa 기반의 쿼리 시스템을 사용합니다:

RootDatabase 구조

```
#[salsa::db]
pub struct RootDatabase {
storage: salsa::Storage<RootDatabase>,
}
```

컴파일 단계별 데이터베이스 그룹

1. **FilesGroup**: 파일 시스템 인터페이스
2. **ParserGroup**: 파싱 관련 쿼리
3. **SemanticGroup**: 의미 분석 쿼리
4. **LoweringGroup**: Lowering 관련 쿼리
5. **SierraGenGroup**: Sierra 생성 쿼리

쿼리 기반 컴파일

```
trait Database:
```

```
ParserGroup +

DefsGroup +

SemanticGroup +

LoweringGroup +

SierraGenGroup

{

// 각 그룹은 특정 컴파일 단계의 쿼리들을 정의

}
```

병렬 처리와 성능 최적화

데이터베이스 워밍업

컴파일러는 병렬 처리를 통한 성능 최적화를 지원합니다:

```
pub enum DbWarmupContext {

Warmup { pool: ThreadPool },

NoWarmup,

}
```

병렬 진단 처리

```
fn warmup_diagnostics_blocking(db: &dyn LoweringGroup, crates: Vec<CrateInput>)

let _: () = par_map(db, crates, |db, crate_input| {
```

```
let crate_id = crate_input.into_crate_long_id(db).intern(db);

let crate_modules = db.crate_modules(crate_id);

// 병렬로 각 모듈의 진단 정보 계산

});

}
```

에러 처리와 진단

진단 시스템

```
struct DiagnosticsReporter {

diagnostics: Vec<Diagnostic>,

severity_filter: Severity,

}
```

다단계 에러 수집

1. **파싱 에러**: 구문 오류와 복구
2. **의미 분석 에러**: 타입 오류, 스코프 오류
3. **Lowering 에러**: 변환 과정의 오류
4. **Sierra 생성 에러**: IR 생성 오류
5. **CASM 생성 에러**: 어셈블리 생성 오류

특수 컴파일 모드

1. Starknet 컨트랙트 컴파일

`starknet-compile` 바이너리는 스마트 컨트랙트 전용 컴파일을 제공합니다:

- 컨트랙트 클래스 생성
- ABI 정보 포함
- 스토리지 변수 처리
- 이벤트 정의 처리

2. Sierra 전용 컴파일

`sierra-compile` 바이너리는 Sierra IR을 CASM으로 직접 컴파일합니다:

- 기존 Sierra 프로그램 입력
- 메타데이터 계산
- CASM 생성 및 최적화

컴파일 파이프라인의 확장성

플러그인 시스템

Cairo 컴파일러는 플러그인을 통한 확장을 지원합니다:

- 커스텀 매크로 정의
- 코드 생성 플러그인
- 분석 도구 통합

외부 도구 통합

1. **cairo-format**: 코드 포매팅
2. **cairo-test**: 테스트 실행

3. **cairo-run**: 프로그램 실행

4. **cairo-size-profiler**: 크기 분석

결론

Cairo 컴파일 파이프라인은 복잡하지만 체계적으로 설계된 다단계 변환 과정입니다. 각 단계는 명확한 책임을 가지며, Salsa 기반의 쿼리 시스템을 통해 효율적인 증분 컴파일을 지원합니다. 이러한 설계는 Cairo 언어의 특수한 요구사항인 증명 가능한 계산과 가스 효율성을 모두 만족시키면서도, 개발자 친화적인 컴파일 경험을 제공합니다.